Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет по лабораторной работе № 1

Тема: «Интегрированная среда разработки ПО»

Вариант 22

Выполнил:

студент группы 434701

Юденко Е.А.

Проверил:

Внук О.М.

Минск, 2026

ВВЕДЕНИЕ

Лабораторная работа посвящена знакомству с интегрированной средой разработки программного обеспечения, созданию приложения с графическим пользовательским интерфейсом на объектно–ориентированном языке C++ и изучению архитектурного шаблона Model–View–Controller с активной моделью. В соответствии с методическими указаниями в работе необходимо разработать приложение по заданному варианту, продемонстрировать основные возможности выбранной среды разработки, описать структуру MVC и выполнить тестирование.

В качестве практического задания выбран вариант 22 – утилита «Сколько раз ты сказал “потом”». Программа по введенному возрасту выполняет оценку приблизительного числа случаев, когда человек откладывал дела на потом, с использованием принятой в программе усредненной статистики. В приложении реализованы главное окно и отдельное окно ввода данных, восстановление последнего значения, проверка корректности ввода и обновление результата.

Целью работы является разработка оконного приложения на C++ без использования сторонних библиотек графического интерфейса, а также практическое применение MVC с активной моделью.

Для решения задачи выбраны Microsoft Visual Studio Code в сочетании с компилятором MSVC и системным интерфейсом WinAPI. Такой вариант позволяет использовать объектно–ориентированный C++ и стандартные средства Windows для построения графического интерфейса.

1 УСЛОВИЕ ЛАБОРАТОРНОЙ РАБОТЫ

Разработать утилиту «Сколько раз ты сказал “потом”». По введенному возрасту программа должна оценивать примерное количество раз, когда человек откладывал дела на потом, используя усредненную статистику. На главной форме должна находиться кнопка «Ввести данные». По нажатию на кнопку должно открываться дополнительное окно для ввода информации. Последние введенные данные необходимо запоминать в главном окне и восстанавливать при повторном вводе. После этого на главной форме требуется отображать результат – вычисленное значение. При вводе некорректных данных необходимо выводить сообщение об ошибке. В приложение требуется добавить каркас MVC с активной моделью.

В методических указаниях также предусмотрены варианты с пассивной моделью и вариант, в котором контроллер совмещается с пассивной моделью. В настоящем отчете реализован основной требуемый вариант с активной моделью.

Язык программирования – C++. Среда разработки – Microsoft Visual Studio Code. Для построения оконного интерфейса применен WinAPI. В пользовательском интерфейсе используются Unicode–вызовы WinAPI, что обеспечивает корректное отображение русскоязычных подписей.

2 ОПИСАНИЕ АЛГОРИТМОВ И РЕШЕНИЙ

2.1 Исходные данные и допущения

Исходными данными программы является возраст человека A, задаваемый целым числом лет. Вводится значение от 1 до 120 лет. Граница диапазона выбрана для исключения очевидно некорректных значений и одновременно не ограничивает типичные варианты использования приложения.

Так как методические указания требуют использовать усредненную статистику, в учебной модели принято допущение о двух отложенных делах в день. Данное значение является параметром модели, необходимым для выполнения учебного задания, и не рассматривается как достоверное статистическое утверждение о поведении людей.

$$N = A \cdot 365 \cdot 2, \quad (2.1)$$

где N – оценочное количество случаев откладывания дел на потом; A – возраст человека, лет; 365 – принятое количество дней в году; 2 – усредненное в рамках учебной модели количество случаев в день.

2.2 Алгоритм работы программы

Программа запускает главное окно, в котором размещены кнопка «Ввести данные» и метки с последними введенными данными и результатом. При нажатии кнопки контроллер создает дополнительное окно ввода. Если возраст уже вводился ранее, его значение передается в окно ввода и автоматически восстанавливается.

После подтверждения ввода выполняется проверка данных. Пустая строка рассматривается как ошибка. Далее проверяется принадлежность значения заданному диапазону. После успешной проверки возраст передается модели. Модель вычисляет значение N по формуле (2.1), сохраняет возраст и результат и инициирует уведомление об изменении состояния. Представление получает новые значения и обновляет метки главного окна.

При ошибке дополнительное окно остается открытым, пользователю выводится сообщение с пояснением причины ошибки, после чего можно повторить ввод. При успешном вводе окно закрывается. Последовательность действий программы состоит из запуска, открытия окна ввода, восстановления последнего возраста, проверки значения, расчета результата и обновления главного окна.

2.3 Реализация активной модели MVC

Архитектура приложения построена по шаблону MVC. Модель содержит состояние предметной области и выполняет расчет. Представление отвечает за создание окон, элементов управления и отображение результата. Контроллер обрабатывает действия пользователя и передает данные модели. Активность модели реализована через функцию обратного вызова onChanged: после успешного изменения возраста модель самостоятельно уведомляет представление о необходимости обновления. Общая схема взаимодействия имеет вид: пользователь –> View –> Controller –> Model –> onChanged –> View.

Связь компонентов построена следующим образом: пользователь взаимодействует с представлением; представление передает событие контроллеру; контроллер вызывает метод модели setAge(); после изменения данных модель вызывает callback; callback обращается к методам представления updateData() и updateResult(). Таким образом, модель не зависит от конкретного элемента пользовательского интерфейса, а представление получает данные через контроллерную связь с моделью.

3 ОПИСАНИЕ ФУНКЦИОНАЛЬНОЙ СТРУКТУРЫ ПРОЕКТА

Программа включает модель, представление, контроллер и отдельное окно ввода. Точка входа wWinMain регистрирует класс главного окна, создает его и запускает цикл обработки сообщений Windows.

Таблица 3.1 – Основные классы программы

Класс	Назначение	Основные элементы
Model	Хранение возраста и расчет результата.	age, result, setAge(), getAge(), getResult(), onChanged
MainView	Отображение главного окна и результата.	Кнопка, метки, updateData(), updateResult(), showError()
InputWindow	Ввод и проверка возраста.	EDIT, кнопки OK/Отмена, обработчик WM_COMMAND
Controller	Связь View и Model, обработка действий.	initialize(), openInputWindow(), callback модели

3.1 Класс Model

Класс Model является активной моделью. Он хранит age и result. Метод setAge() проверяет диапазон, записывает новое состояние, выполняет расчет и вызывает callback onChanged. Методы getAge() и getResult() возвращают текущее состояние.

3.2 Представление и окно ввода

Класс MainView отвечает за главное окно. В нем создаются кнопка «Ввести данные», метка последних данных и метка результата. Метод updateData() формирует строку с возрастом, а метод updateResult() формирует строку с оценкой количества случаев откладывания дел.

Класс InputWindow реализует дополнительное окно. Для ввода используется поле EDIT, для подтверждения – кнопка OK, для закрытия без изменения – кнопка «Отмена». При создании окна передается последнее значение возраста; если оно существует, поле заполняется автоматически. Это обеспечивает требуемое восстановление последних введенных данных.

3.3 Класс Controller

Класс Controller хранит Model и MainView и последнее введенное значение lastAge. Метод initialize() связывает активную модель с callback–

функцией, а openInputWindow() открывает окно ввода. После успешного ввода новое значение записывается в модель и lastAge.

4 ПОРЯДОК СБОРКИ И ИСПОЛЬЗОВАНИЯ

Для выполнения лабораторной работы используется Microsoft Visual Studio Code с установленным набором средств для разработки на C++. Проект содержит исходный файл main.cpp. Для анализа кода могут применяться подсветка синтаксиса, автодополнение, переход к объявлениям и средствам отладки. Для демонстрации архитектуры проекта могут использоваться расширения для построения UML и SysMLv2.

Сборка проекта при использовании компилятора MSVC выполняется из терминала разработчика командой:

```
cl /EHsc /std:c++17 main.cpp user32.lib gdi32.lib /link /SUBSYSTEM:WINDOWS
```

Библиотека user32.lib требуется для функций создания окон, вывода сообщений и управления текстом элементов интерфейса. В исходном коде также могут использоваться директивы #pragma comment(lib, ...) для автоматического подключения системных библиотек при сборке средствами MSVC.

После успешной сборки формируется исполняемый файл. При запуске отображается главное окно. Нажатие кнопки «Ввести данные» открывает дополнительное окно. После ввода корректного возраста программа закрывает окно ввода и отображает сохраненное значение и расчетный результат на главной форме.

Для демонстрации возможностей отладчика достаточно установить точку останова в Model::setAge(), запустить программу в режиме отладки и проверить изменение переменных age и result. После выполнения callback можно наблюдать переход к updateData() и updateResult(), что наглядно показывает взаимодействие активной модели и представления.

5 ОПИСАНИЕ МЕТОДА ТЕСТИРОВАНИЯ И РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Тестирование направлено на проверку корректности расчета, обработки ошибочного ввода и восстановления последних введенных данных. Основой тестирования являются заранее определенные граничные и типовые значения возраста.

Таблица 5.1 – Тестовые случаи

№	Исходные данные	Ожидаемый результат
1	Возраст 20	20 × 365 × 2 = 14600; значение отображается на главной форме
2	Возраст 1	1 × 365 × 2 = 730
3	Возраст 120	120 × 365 × 2 = 87600
4	Возраст 0	Сообщение об ошибке; данные не принимаются
5	Возраст 121	Сообщение об ошибке; данные не принимаются
6	Пустое поле	Сообщение «Введите возраст»
7	Повторное открытие окна после возраста 20	В поле ввода автоматически отображается 20
8	Нажатие «Отмена»	Окно закрывается без изменения текущих данных

Для возраста 20 лет результат составляет 14 600 случаев, для граничных допустимых значений 1 и 120 лет – 730 и 87 600 соответственно. Значения 0 и 121 отклоняются, повторное открытие окна восстанавливает последний возраст. После успешного setAge() вызывается onChanged(), поэтому проверяется также требование активной модели MVC.

6 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

6.1 Какие компоненты входят в состав IDE?

В состав интегрированной среды разработки обычно входят редактор исходного кода, средства управления проектом, компилятор или интеграция с компилятором, компоновщик, отладчик, терминал, средства поиска и навигации, автодополнение, диагностика ошибок, система расширений и интеграция с системами контроля версий.

6.2 Каковы основные приемы работы с редактором кода?

К основным приемам относятся набор и редактирование текста программы, переход к определениям и объявлениям, автоматическое завершение конструкций, поиск и замена, многоуровневая навигация, форматирование, переименование идентификаторов, работа с диагностическими сообщениями компилятора и использование подсказок и контекстного анализа.

6.3 Каким образом выполняется сборка и запуск проекта?

Исходный файл C++ проходит компиляцию, в результате которой формируется объектный код. Затем компоновщик объединяет объектный код и необходимые библиотеки, после чего формируется исполняемый файл. В рассматриваемой программе к проекту подключаются системные библиотеки user32.lib и gdi32.lib. Запуск может выполняться непосредственно из среды разработки или из терминала; при использовании отладчика программа запускается под его управлением.

6.4 Что такое MVC?

MVC – архитектурный шаблон, разделяющий приложение на модель, представление и контроллер. Модель хранит данные и реализует предметную логику, представление отображает состояние программы и интерфейс, контроллер обрабатывает действия пользователя и организует взаимодействие представления с моделью.

6.5 Чем отличается активная модель от пассивной?

Активная модель самостоятельно уведомляет зависимые компоненты об изменении состояния. Пассивная модель этого не делает, поэтому обновление представления выполняет контроллер. В работе реализована активная модель через onChanged().

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была разработана оконная утилита «Сколько раз ты сказал “потом”» на языке C++ с использованием WinAPI. Программа принимает возраст человека, оценивает примерное количество случаев откладывания дел и отображает результат на главной форме.

В приложении реализованы все функциональные требования варианта 22: отдельное окно ввода, восстановление последнего введенного возраста, проверка корректности данных и вывод сообщения об ошибке. Для расчета использована учебная модель с коэффициентом два случая в день.

Архитектура программы построена по шаблону MVC. Модель содержит данные и вычислительную логику, представление отвечает за графический интерфейс, контроллер связывает действия пользователя с моделью. Для реализации активной модели применен callback, благодаря которому после изменения состояния модель инициирует обновление представления.

Кроме того, рассмотрены средства интегрированной среды разработки и порядок сборки проекта с использованием MSVC. Сформирован набор тестов для типовых, граничных и ошибочных входных данных, а также рассмотрен сценарий пошаговой отладки.

ПРИЛОЖЕНИЕ А

Листинг программы

```
#define UNICODE
#define _UNICODE

#include <windows.h>
#include <string>
#include <functional>
#include <stdexcept>
#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")

// ====================================================
// MODEL
// ====================================================

class Model
{
private:
    int age = 0;
    long long result = 0;

    //   :
    // 2       " ".
    static constexpr int POSTPONEMENTS_PER_DAY = 2;

    std::function<void()> onChanged;

public:
    void setOnChanged(std::function<void()> callback)
    {
        onChanged = callback;
    }

    bool setAge(int newAge)
    {
        if (newAge < 1 || newAge > 120)
            return false;

        age = newAge;

        result =
            static_cast<long long>(age) *
            365LL *
            POSTPONEMENTS_PER_DAY;

        //     .
        if (onChanged)
            onChanged();

        return true;
    }

    int getAge() const
    {
        return age;
    }

    long long getResult() const
    {
        return result;
    }
};
```

```
// ====================================================
// VIEW
// ====================================================

class MainView
{
private:
    HWND hwnd = nullptr;
    HWND buttonInput = nullptr;
    HWND labelData = nullptr;
    HWND labelResult = nullptr;

public:
    void setWindow(HWND window)
    {
        hwnd = window;
    }

    HWND getWindow() const
    {
        return hwnd;
    }

    void createControls()
    {
        buttonInput = CreateWindowW(
            L"BUTTON",
            L"\u0412\u0432\u0435\u0441\u0442\u0438 \u0434\u0430\u043d\u043d\u044b\u0435",
            WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
            30, 30, 180, 40,
            hwnd,
            reinterpret_cast<HMENU>(1001),
            GetModuleHandleW(nullptr),
            nullptr
        );

        labelData = CreateWindowW(
            L"STATIC",
            L"\u041f\u043e\u0441\u043b\u0435\u0434\u043d\u0438\u0435
\u0434\u0430\u043d\u043d\u044b\u0435: \u043d\u0435\u0442",
            WS_VISIBLE | WS_CHILD | SS_LEFT,
            30, 90, 520, 25,
            hwnd,
            nullptr,
            GetModuleHandleW(nullptr),
            nullptr
        );

        labelResult = CreateWindowW(
            L"STATIC",
            L"\u0420\u0435\u0437\u0443\u043b\u044c\u0442\u0430\u0442: \u043d\u0435\u0442",
            WS_VISIBLE | WS_CHILD | SS_LEFT,
            30, 125, 520, 45,
            hwnd,
            nullptr,
            GetModuleHandleW(nullptr),
            nullptr
        );
    }

    void updateData(int age)
    {
        std::wstring text =
            L"\u041f\u043e\u0441\u043b\u0435\u0434\u043d\u0438\u0435
\u0434\u0430\u043d\u043d\u044b\u0435: \u0432\u043e\u0437\u0440\u0430\u0441\u0442 = " +
            std::to_wstring(age);

        SetWindowTextW(labelData, text.c_str());
    }
```

```
    void updateResult(long long result)
    {
        std::wstring text =
            L"\u041f\u0440\u0438\u043c\u0435\u0440\u043d\u043e\u0435
\u043a\u043e\u043b\u0438\u0447\u0435\u0441\u0442\u0432\u043e \u0440\u0430\u0437,
\u043a\u043e\u0433\u0434\u0430 \u0447\u0435\u043b\u043e\u0432\u0435\u043a
\u0441\u043a\u0430\u0437\u0430\u043b \u00ab\u043f\u043e\u0442\u043e\u043c\u00bb: " +
            std::to_wstring(result);

        SetWindowTextW(labelResult, text.c_str());
    }

    void showError(const std::wstring& message)
    {
        MessageBoxW(
            hwnd,
            message.c_str(),
            L"\u041e\u0448\u0438\u0431\u043a\u0430",
            MB_OK | MB_ICONERROR
        );
    }
};

// ====================================================
// INPUT VIEW
// ====================================================

class InputWindow
{
private:
    HWND hwnd = nullptr;
    HWND editAge = nullptr;
    HWND parent = nullptr;

    std::function<void(int)> onOk;

public:
    InputWindow(
        HWND parentWindow,
        int lastAge,
        std::function<void(int)> callback)
        : parent(parentWindow), onOk(callback)
    {
        WNDCLASSW wc{};

        wc.lpfnWndProc = InputWindow::WndProc;
        wc.hInstance = GetModuleHandleW(nullptr);
        wc.lpszClassName = L"InputWindowClass";
        wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
        wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);

        //      .
        RegisterClassW(&wc);

        hwnd = CreateWindowW(
            L"InputWindowClass",
            L"\u0412\u0432\u043e\u0434 \u0434\u0430\u043d\u043d\u044b\u0445",
            WS_CAPTION | WS_SYSMENU | WS_MINIMIZEBOX,
            CW_USEDEFAULT,
            CW_USEDEFAULT,
            350,
            220,
            parent,
            nullptr,
            GetModuleHandleW(nullptr),
            this
        );
```

```
    if (hwnd == nullptr)
    {
        return;
    }

    ShowWindow(hwnd, SW_SHOW);
    UpdateWindow(hwnd);

    //    .
    if (lastAge > 0 && editAge != nullptr)
    {
        SetWindowTextW(
            editAge,
            std::to_wstring(lastAge).c_str()
        );

        //    .
        SendMessageW(editAge, EM_SETSEL, 0, -1);
        SetFocus(editAge);
    }
}

HWND getWindow() const
{
    return hwnd;
}

static LRESULT CALLBACK WndProc(
    HWND hwnd,
    UINT message,
    WPARAM wParam,
    LPARAM lParam)
{
    InputWindow* window =
        reinterpret_cast<InputWindow*>(
            GetWindowLongPtrW(hwnd, GWLP_USERDATA)
        );

    if (message == WM_NCCREATE)
    {
        auto* createStruct =
            reinterpret_cast<CREATESTRUCTW*>(lParam);

        window =
            reinterpret_cast<InputWindow*>(
                createStruct->lpCreateParams
            );

        SetWindowLongPtrW(
            hwnd,
            GWLP_USERDATA,
            reinterpret_cast<LONG_PTR>(window)
        );
    }

    if (window == nullptr)
    {
        return DefWindowProcW(
            hwnd, message, wParam, lParam
        );
    }

    switch (message)
    {
    case WM_CREATE:
    {
        CreateWindowW(
            L"STATIC",
```

```
                L"\u0412\u0432\u0435\u0434\u0438\u0442\u0435
\u0432\u043e\u0437\u0440\u0430\u0441\u0442:",
                WS_VISIBLE | WS_CHILD | SS_LEFT,
                30, 25, 200, 25,
                hwnd,
                nullptr,
                GetModuleHandleW(nullptr),
                nullptr
            );

            window->editAge = CreateWindowW(
                L"EDIT",
                L"",
                WS_VISIBLE | WS_CHILD |
                WS_BORDER | ES_NUMBER | ES_AUTOHSCROLL,
                30, 55, 270, 30,
                hwnd,
                reinterpret_cast<HMENU>(2001),
                GetModuleHandleW(nullptr),
                nullptr
            );

            CreateWindowW(
                L"BUTTON",
                L"OK",
                WS_VISIBLE | WS_CHILD | BS_DEFPUSHBUTTON,
                70, 105, 90, 35,
                hwnd,
                reinterpret_cast<HMENU>(2002),
                GetModuleHandleW(nullptr),
                nullptr
            );

            CreateWindowW(
                L"BUTTON",
                L"\u041e\u0442\u043c\u0435\u043d\u0430",
                WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
                180, 105, 90, 35,
                hwnd,
                reinterpret_cast<HMENU>(2003),
                GetModuleHandleW(nullptr),
                nullptr
            );

            return 0;
        }

        case WM_COMMAND:
        {
            const int id = LOWORD(wParam);

            if (id == 2002)
            {
                wchar_t buffer[100]{};

                GetWindowTextW(
                    window->editAge,
                    buffer,
                    100
                );

                std::wstring input(buffer);

                if (input.empty())
                {
                    MessageBoxW(
                        hwnd,
                        L"\u0412\u0432\u0435\u0434\u0438\u0442\u0435
\u0432\u043e\u0437\u0440\u0430\u0441\u0442.",
```

```
                        L"\u041e\u0448\u0438\u0431\u043a\u0430",
                        MB_OK | MB_ICONERROR
                    );

                    SetFocus(window->editAge);
                    return 0;
                }

                // ,    .
                if (input.find_first_not_of(L"0123456789") != std::wstring::npos)
                {
                    MessageBoxW(
                        hwnd,
                        L"\u0412\u043e\u0437\u0440\u0430\u0441\u0442
\u0434\u043e\u043b\u0436\u0435\u043d \u0431\u044b\u0442\u044c \u0446\u0435\u043b\u044b\u043c
\u0447\u0438\u0441\u043b\u043e\u043c.",
                        L"\u041e\u0448\u0438\u0431\u043a\u0430",
                        MB_OK | MB_ICONERROR
                    );

                    SetFocus(window->editAge);
                    return 0;
                }

                int age = 0;

                try
                {
                    size_t processed = 0;
                    age = std::stoi(input, &processed);

                    if (processed != input.size())
                    {
                        throw std::invalid_argument("invalid input");
                    }
                }
                catch (...)
                {
                    MessageBoxW(
                        hwnd,
                        L"\u041d\u0435\u043a\u043e\u0440\u0440\u0435\u043a\u0442\u043d\u044b\u
0435 \u0434\u0430\u043d\u043d\u044b\u0435.",
                        L"\u041e\u0448\u0438\u0431\u043a\u0430",
                        MB_OK | MB_ICONERROR
                    );

                    SetFocus(window->editAge);
                    return 0;
                }

                if (age < 1 || age > 120)
                {
                    MessageBoxW(
                        hwnd,
                        L"\u0412\u0432\u0435\u0434\u0438\u0442\u0435
\u0432\u043e\u0437\u0440\u0430\u0441\u0442 \u043e\u0442 1 \u0434\u043e 120
\u043b\u0435\u0442.",
                        L"\u041e\u0448\u0438\u0431\u043a\u0430",
                        MB_OK | MB_ICONERROR
                    );

                    SetFocus(window->editAge);
                    return 0;
                }

                if (window->onOk)
                {
                    window->onOk(age);
                }
```

```
                DestroyWindow(hwnd);
                return 0;
            }

            if (id == 2003)
            {
                DestroyWindow(hwnd);
                return 0;
            }

            break;
        }

        case WM_CLOSE:
            DestroyWindow(hwnd);
            return 0;

        case WM_NCDESTROY:
        {
            SetWindowLongPtrW(hwnd, GWLP_USERDATA, 0);
            LRESULT result = DefWindowProcW(
                hwnd, message, wParam, lParam
            );
            delete window;
            return result;
        }
        }

        return DefWindowProcW(
            hwnd, message, wParam, lParam
        );
    }
};

// ====================================================
// CONTROLLER
// ====================================================

class Controller
{
private:
    Model model;
    MainView view;

    //   .
    int lastAge = 0;

public:
    void initialize(HWND hwnd)
    {
        view.setWindow(hwnd);
        view.createControls();

        //     View.
        model.setOnChanged([this]()
        {
            view.updateData(model.getAge());
            view.updateResult(model.getResult());
        });
    }

    void openInputWindow()
    {
        new InputWindow(
            view.getWindow(),
            lastAge,

            [this](int age)
```

```
            {
                if (!model.setAge(age))
                {
                    view.showError(
                        L"\u041d\u0435\u043a\u043e\u0440\u0440\u0435\u043a\u0442\u043d\u044b\u
0435 \u0434\u0430\u043d\u043d\u044b\u0435."
                    );
                    return;
                }

                lastAge = age;
            }
        );
    }
};

// ====================================================
//
// ====================================================

Controller* controller = nullptr;

LRESULT CALLBACK MainWndProc(
    HWND hwnd,
    UINT message,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (message)
    {
    case WM_CREATE:
    {
        controller = new Controller();
        controller->initialize(hwnd);
        return 0;
    }

    case WM_COMMAND:
    {
        if (LOWORD(wParam) == 1001)
        {
            controller->openInputWindow();
            return 0;
        }

        break;
    }

    case WM_DESTROY:
    {
        delete controller;
        controller = nullptr;

        PostQuitMessage(0);
        return 0;
    }
    }

    return DefWindowProcW(
        hwnd, message, wParam, lParam
    );
}

// ====================================================
//
// ====================================================

int WINAPI wWinMain(
    HINSTANCE hInstance,
```

```
    HINSTANCE,
    PWSTR,
    int nCmdShow)
{
    WNDCLASSW wc{};

    wc.lpfnWndProc = MainWndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = L"MainWindowClass";
    wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);

    if (!RegisterClassW(&wc))
    {
        MessageBoxW(
            nullptr,
            L"\u041d\u0435 \u0443\u0434\u0430\u043b\u043e\u0441\u044c
\u0437\u0430\u0440\u0435\u0433\u0438\u0441\u0442\u0440\u0438\u0440\u043e\u0432\u0430\u0442\u04
4c \u043e\u043a\u043d\u043e.",
            L"\u041e\u0448\u0438\u0431\u043a\u0430",
            MB_OK | MB_ICONERROR
        );

        return 1;
    }

    HWND hwnd = CreateWindowW(
        L"MainWindowClass",
        L"\u0421\u043a\u043e\u043b\u044c\u043a\u043e \u0440\u0430\u0437 \u0442\u044b
\u0441\u043a\u0430\u0437\u0430\u043b \u00ab\u043f\u043e\u0442\u043e\u043c\u00bb",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT,
        CW_USEDEFAULT,
        620,
        280,
        nullptr,
        nullptr,
        hInstance,
        nullptr
    );

    if (hwnd == nullptr)
    {
        MessageBoxW(
            nullptr,
            L"\u041d\u0435 \u0443\u0434\u0430\u043b\u043e\u0441\u044c
\u0441\u043e\u0437\u0434\u0430\u0442\u044c \u0433\u043b\u0430\u0432\u043d\u043e\u0435
\u043e\u043a\u043d\u043e.",
            L"\u041e\u0448\u0438\u0431\u043a\u0430",
            MB_OK | MB_ICONERROR
        );

        return 1;
    }

    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG message{};

    while (GetMessageW(&message, nullptr, 0, 0) > 0)
    {
        TranslateMessage(&message);
        DispatchMessageW(&message);
    }

    return static_cast<int>(message.wParam);
}
```